Faculty of Engineering & Technology

Electrical & Computer Engineering Department

Computer Architecture

ENCS4370

Project Two – (Processor Architecture)

Prepared by:

Mohammad Abdallah – 1230096

Khaled Bani Oudeh – 1230571

Motaz Taysir – 1231596

Instructor: **Dr. Aziz.**

Section: **1**

Date: **June 25, 2026**

Abstract

This report presents the design and implementation of a 32-bit pipelined RISC processor using Verilog HDL. The processor supports twenty instructions classified into R-Type, I-Type, and J-Type formats, including arithmetic, logical, memory access, branch, jump, and custom instructions such as CLR, JR, and SWAP. The architecture contains sixteen 32-bit general-purpose registers, a Program Counter (PC), Instruction Memory, Register File, Arithmetic Logic Unit (ALU), Data Memory, Control Unit, Sign/Zero Extension Unit, and a complete five-stage pipeline consisting of Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB).

To improve performance, the processor employs pipelining with separate instruction and data memories, eliminating structural hazards. A Hazard Detection Unit and a Forwarding Unit are integrated into the datapath to minimize pipeline stalls and resolve data dependencies efficiently. The control unit generates all required control signals based on the instruction opcode, enabling correct execution of every supported instruction.

The processor was verified using Verilog simulation and multiple assembly test programs covering all implemented instructions. The generated waveforms confirm the correctness of arithmetic operations, memory accesses, branching, jumping, forwarding, hazard handling, and write-back operations. The final implementation demonstrates a complete and efficient pipelined processor that satisfies all project requirements while maintaining a modular and scalable design.

Table of Contents

List of Figures

List of tables

1. Introduction

Modern computer processors rely on pipelining techniques to improve execution speed and increase instruction throughput. Instead of executing one instruction at a time, pipelined processors divide instruction execution into multiple stages, allowing several instructions to be processed simultaneously. This approach significantly improves processor performance without increasing the clock frequency.

The objective of this project is to design and implement a complete 32-bit pipelined RISC processor using Verilog HDL. The processor follows a five-stage pipeline architecture consisting of Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB). Each stage performs a specific function, enabling multiple instructions to overlap during execution.

The processor supports twenty instructions divided into R-Type, I-Type, and J-Type formats. These include arithmetic, logical, memory, branch, jump, and custom instructions such as CLR, JR, and SWAP. To ensure correct execution, the processor integrates a Control Unit, Hazard Detection Unit, and Forwarding Unit, allowing efficient handling of control and data hazards.

The design was implemented entirely in Verilog HDL using a modular approach. Each hardware component was developed and verified independently before being integrated into the complete processor. Functional verification was performed using simulation waveforms and assembly test programs that cover all supported instructions.

2. Design and Implementation

2.1 Processor Specifications

Before discussing the processor architecture in detail, it is important to summarize the main hardware specifications of the implemented processor. These specifications define the processor organization, supported architecture, memory structure, and overall system capabilities. The processor was designed following the RISC philosophy, where instructions are executed through a simple and efficient five-stage pipeline. Each hardware component was implemented using Verilog HDL and integrated to achieve correct instruction execution while maintaining high performance.

Table 1: Processor Specifications

Feature	Description
Processor Type	32-bit RISC Processor
Pipeline Stages	5
Register File	16 General Purpose Registers
Register Width	32 bits
Instruction Width	32 bits
Data Width	32 bits
Instruction Memory	Separate
Data Memory	Separate
ALU	32-bit
Supported Instructions	20
Hazard Detection	Yes
Forwarding Unit	Yes
Branch Support	Yes
Jump Support	Yes
Custom Instructions	CLR, JR, SWAP

The implemented processor follows a Harvard architecture by separating instruction memory from data memory, allowing instruction fetching and memory access to occur simultaneously without structural conflicts. In addition, forwarding and hazard detection mechanisms were incorporated to reduce pipeline stalls and improve execution efficiency. These features make the processor suitable for executing a variety of arithmetic, logical, memory, and control instructions while maintaining correct pipeline operation.

2.2 Instruction Set Architecture (ISA)

The processor supports twenty instructions that are divided into three instruction formats: R-Type, I-Type, and J-Type. This organization simplifies instruction decoding while providing sufficient functionality to execute arithmetic, logical, memory access, branching, and jump operations. In addition to the standard instruction set, several custom instructions were implemented to satisfy the project requirements and demonstrate the flexibility of the processor design.

Table 2: R-Type Instruction

Instruction	Function
XOR	Bitwise XOR
ADD	Addition
SUB	Subtraction
AND	Bitwise AND
OR	Bitwise OR
NOR	Bitwise NOR
SLL	Shift Left Logical
SRL	Shift Right Logical
CLR	Clear Register

Instruction	Function
XOR	Bitwise XOR
JR	Jump Register

Table 3: I-Type Instructions

Instruction	Function
ADDI	Add Immediate
ANDI	AND Immediate
ORI	OR Immediate
SWAP	Swap Register with Memory
LW	Load Word
SW	Store Word
BEQ	Branch if Equal
BNE	Branch if Not Equal

Table 4: J-Type Instructions

Instruction	Function
J	Jump
JAL	Jump and Link

The R-Type instructions perform register-to-register operations such as arithmetic and logical computations. I-Type instructions are mainly used for immediate values, memory access, and branch operations, while J-Type instructions modify the normal execution flow by changing the Program Counter. The processor also supports custom instructions including CLR, JR, and SWAP, which extend the processor functionality beyond the basic instruction set.

2.3 Processor Architecture

The implemented processor is based on a 32-bit pipelined RISC architecture. The datapath consists of several interconnected hardware modules that work together to execute instructions efficiently. The processor follows a five-stage pipeline, where each instruction progresses through the stages of Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB). Pipeline registers are inserted between consecutive stages to isolate operations and allow multiple instructions to be processed simultaneously.

The processor contains a Program Counter (PC), Instruction Memory, Register File, Arithmetic Logic Unit (ALU), Data Memory, Control Unit, Sign Extension Unit, Hazard Detection Unit, and Forwarding Unit. Each module performs a dedicated function, while control signals coordinate the movement of data throughout the datapath.

Figure 1 illustrates the complete processor datapath used in the implementation.

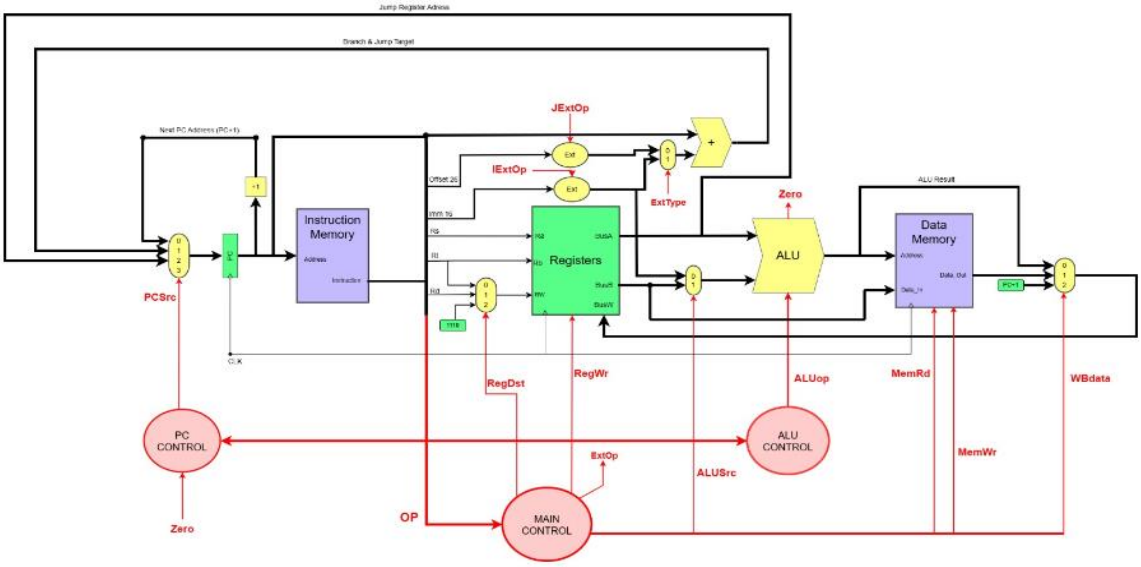


Figure 1: Complete Processor Datapath

2.4 Processor Components

The processor consists of several hardware modules, each responsible for a specific task during instruction execution. The integration of these modules enables the processor to execute instructions correctly while maintaining high performance through pipelining.

a. Program Counter (PC)

The Program Counter stores the address of the next instruction to be executed. During normal execution, the PC is incremented by 1 after every instruction because the instruction memory is word-addressed.

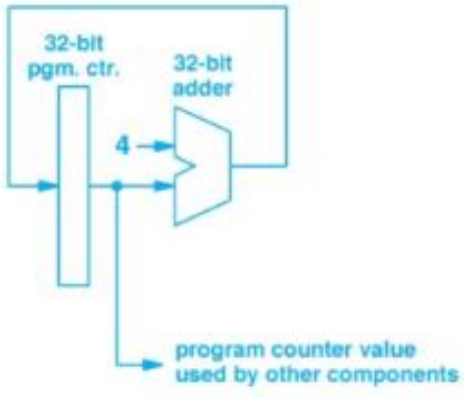


Figure 2: Program Counter (PC)

b. Instruction Memory

Instruction Memory stores all program instructions. During the Instruction Fetch stage, the Program Counter provides the instruction address, and the corresponding instruction is fetched and passed to the IF/ID pipeline register.

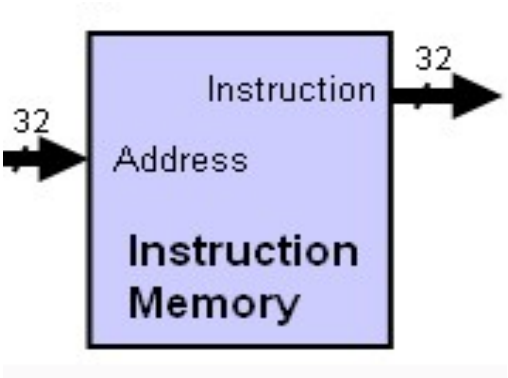


Figure 3: Instruction Memory

c. Register File

The Register File contains sixteen 32-bit general-purpose registers. It provides two read ports and one write port, allowing two operands to be read simultaneously while another register is updated during the Write Back stage.

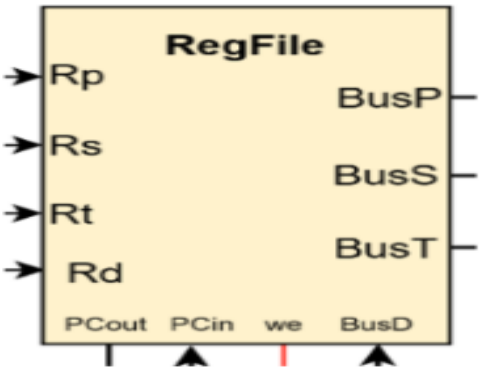


Figure 4: Register File

d. ALU

The ALU performs all arithmetic and logical operations required by the instruction set. Depending on the ALU control signals, it executes operations such as addition, subtraction, AND, OR, NOR, comparison, and address calculation for memory instructions.

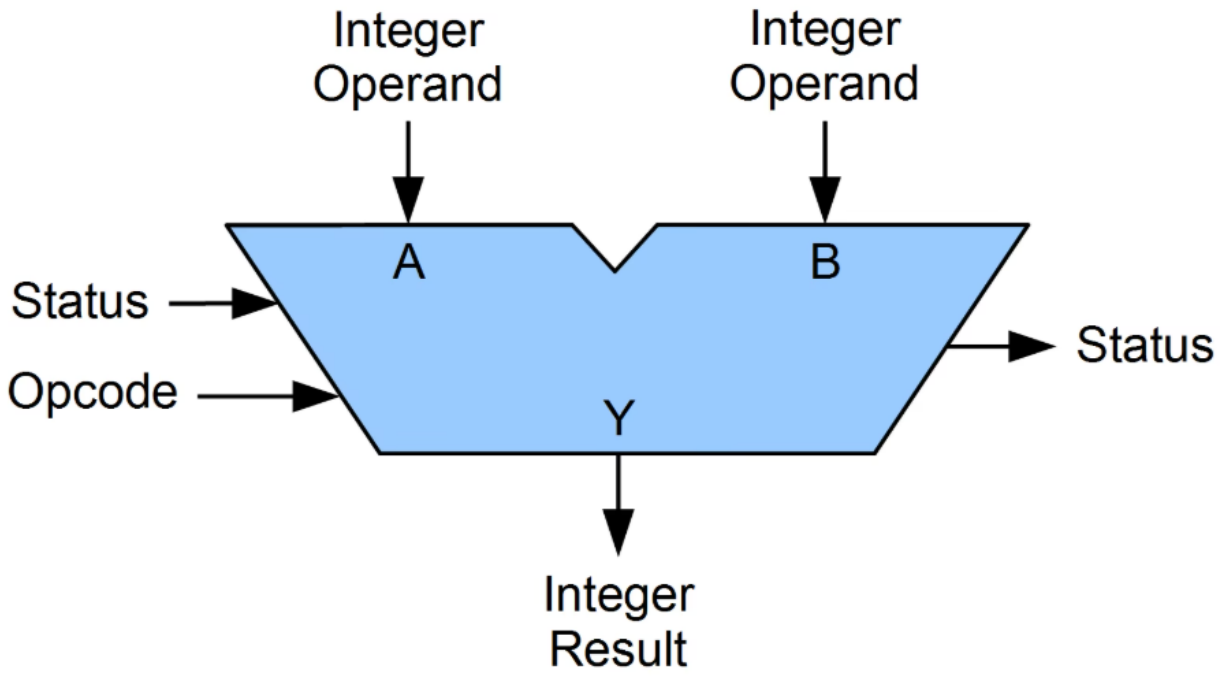


Figure 5: ALU

e. Data Memory

Data Memory is used by load and store instructions. Load instructions read data from memory and write it back into the register file, while store instructions write register values into memory.

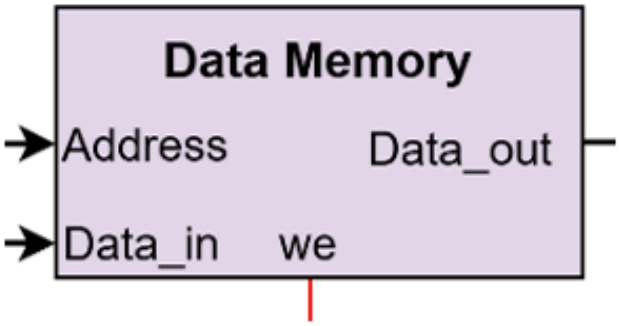


Figure 6: Data Memory

f. Control Unit

The Control Unit decodes the instruction opcode and generates all control signals required for the datapath. These signals determine ALU operations, register writing, memory access, branching, jumping, and data selection.

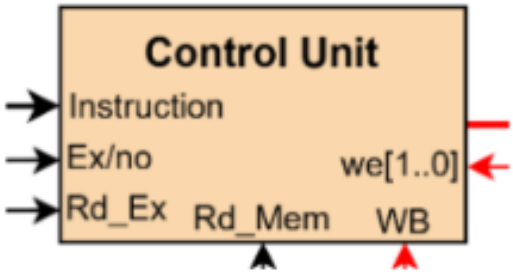


Figure 7: Control Unit

g. Sign/Zero Extension Unit

Extends the 18-bit immediate value to 32 bits. Sign extension is used for arithmetic, memory, and branch instructions, while zero extension is used for logical immediate instructions.

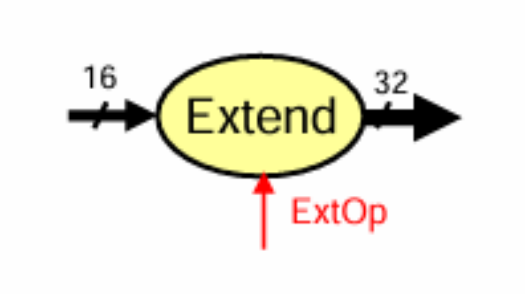


Figure 8: Sign/Zero Extension Unit

2.5 Pipeline Architecture

Pipelining improves processor performance by dividing instruction execution into five independent stages. While one instruction is executed, other instructions are simultaneously fetched, decoded, or accessing memory, resulting in higher instruction throughput.

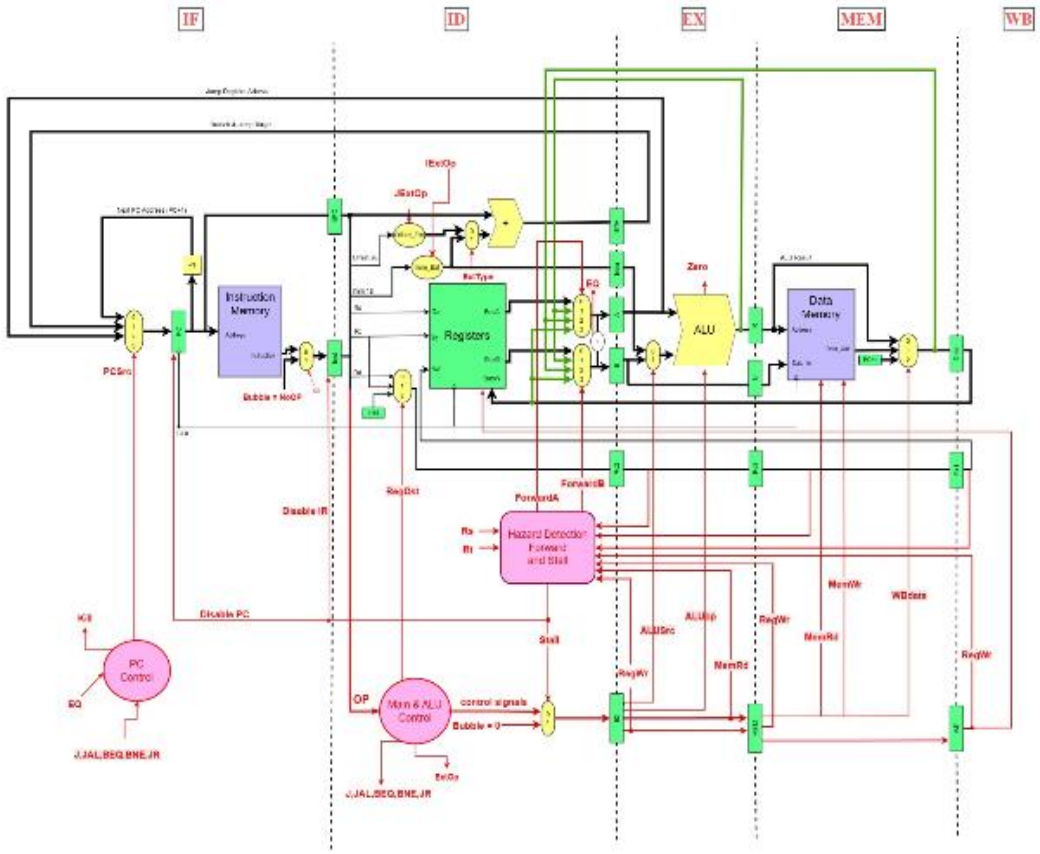


Figure 9: Five-Stage Pipeline Architecture

2.5.1 IF Stage

As shown in Figure 10, the IF stage is responsible for fetching instructions from the Instruction Memory using the address stored in the Program Counter (PC), which is updated every clock cycle either by incrementing by 1 for sequential execution or through the PC Control block for branch and jump instructions, and the fetched 32-bit instruction along with the PC value are stored in the IF/ID pipeline register to be passed to the next stage.

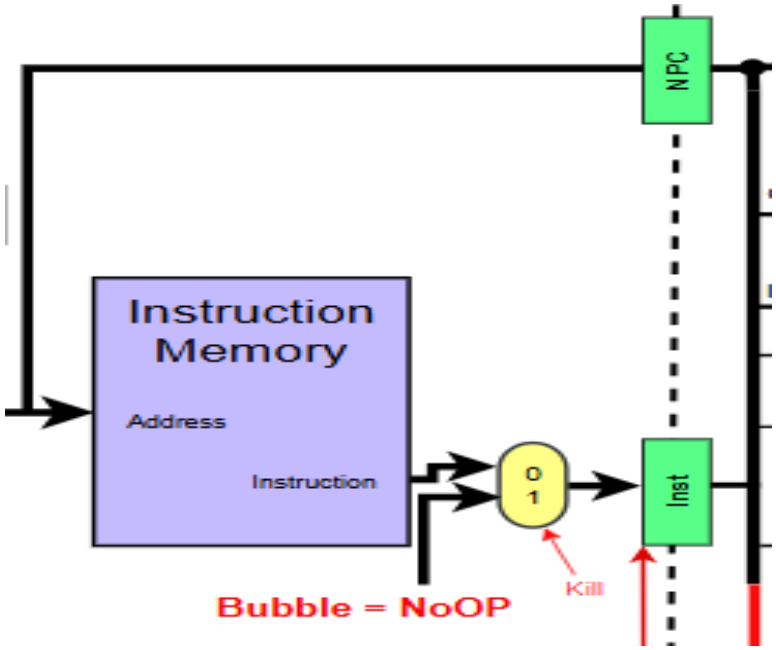


Figure 10 : IF Stage

2.5.2 ID Stage

As shown in Figure 11, the ID stage decodes the 32-bit instruction into its fields (Opcode, Rs, Rt, Rd, Immediate) and the Control Unit generates all necessary control signals based on the Opcode, while the Register File reads the values of the source registers Rs and Rt, and the Sign/Zero Extension Unit extends the Immediate field to 32 bits either by sign extension or zero extension depending on the instruction type, with all decoded values and control signals stored in the ID/EX pipeline register.

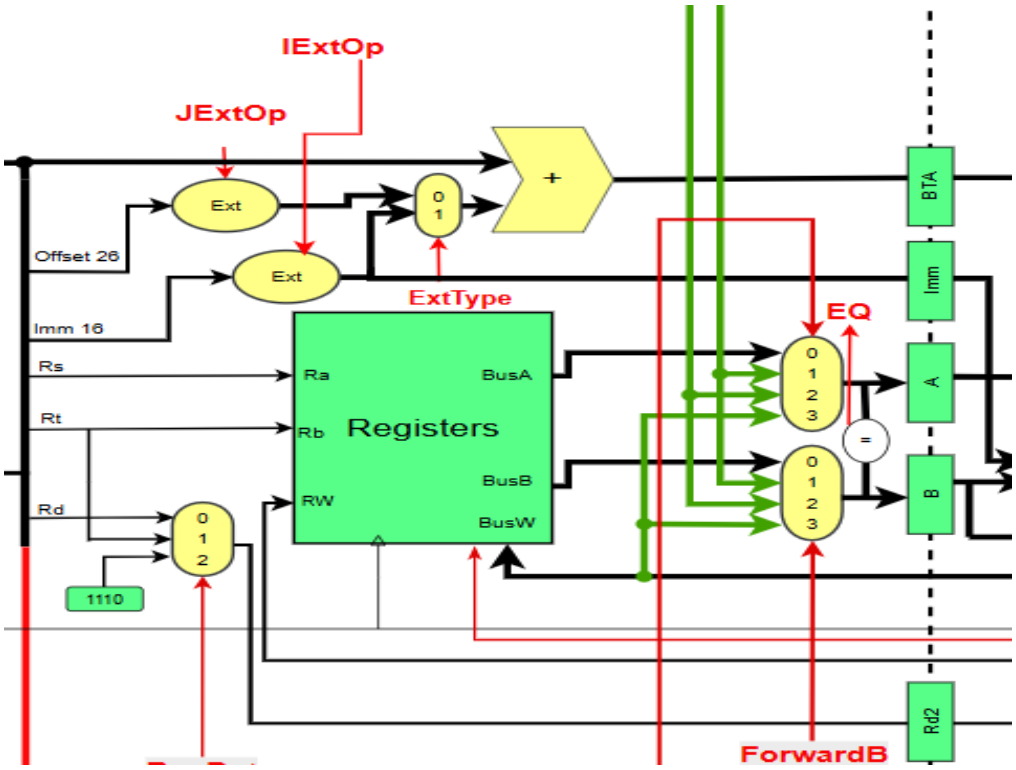


Figure 11 : ID Stage

2.5.3 EX Stage

As shown in Figure 12, the EX stage performs arithmetic and logical operations using the ALU, where the first operand comes from the Rs register and the second operand is selected by the ALUSrc multiplexer (either from the Rt register or the extended Immediate value), and the ALU result is stored in the EX/MEM pipeline register, while for branch instructions (BEQ and BNE) the ALU performs subtraction to evaluate the condition and determine whether the branch is taken.

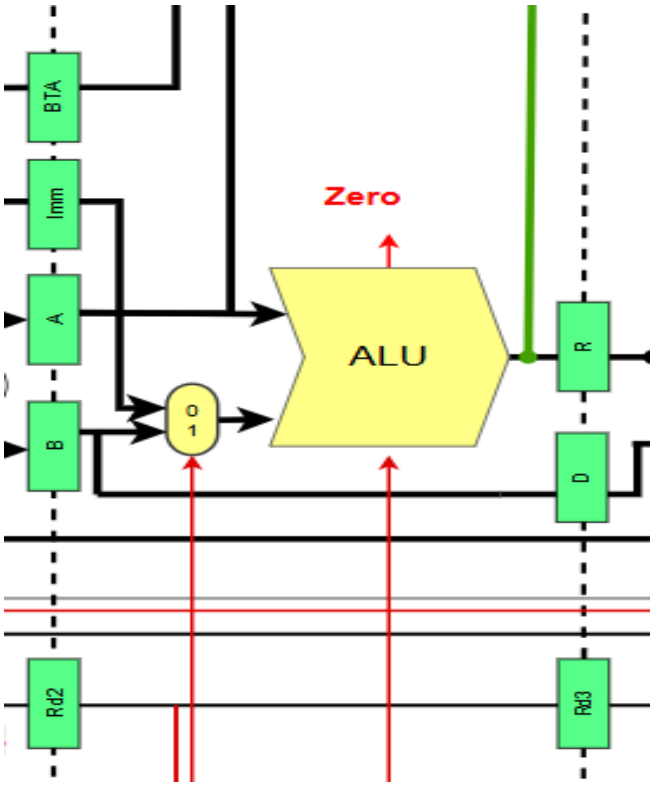


Figure 12 : EX Stage

2.5.4 MEM Stage

As shown in Figure 13, the MEM stage accesses the Data Memory using the ALU result as the memory address, where MemRead is asserted for LW instructions to read data from memory, MemWrite is asserted for SW instructions to write data to memory, and for SWAP both operations are performed simultaneously to exchange the register value with the memory value, with the results stored in the MEM/WB pipeline register.

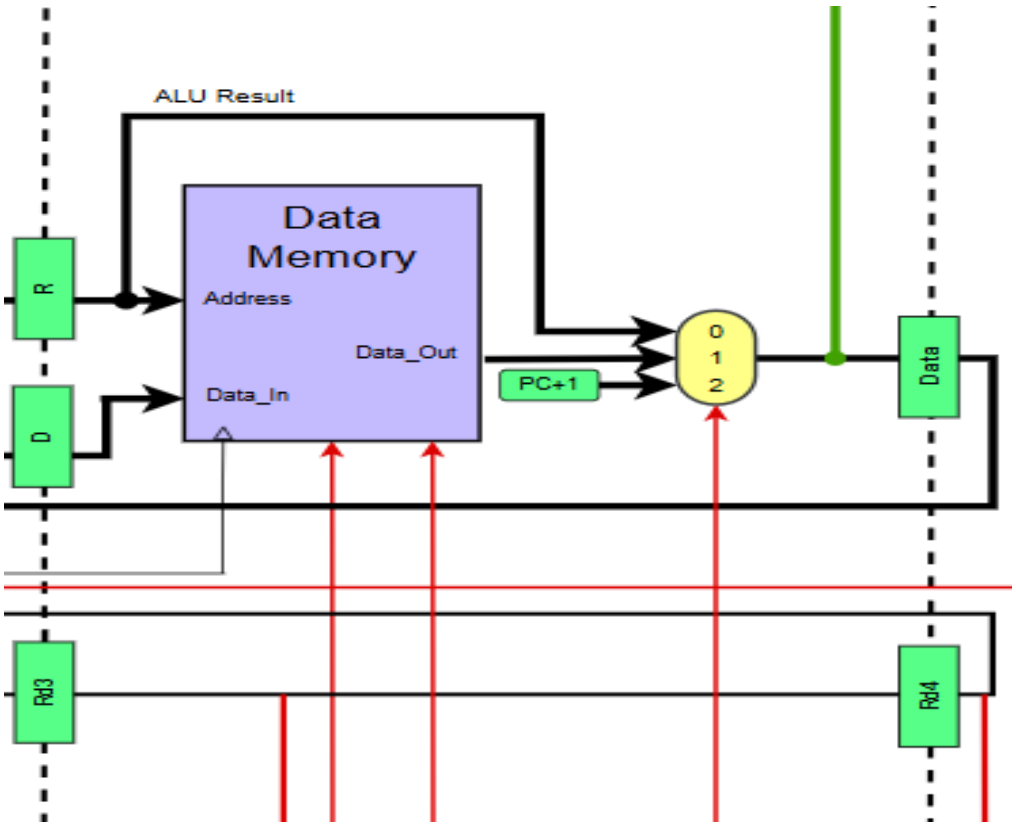


Figure 13 : MEM Stage

2.5.5 WB Stage

As shown in Figure 14, the WB stage writes the final result into the Register File, where MemtoReg selects between the ALU result and the memory read data, RegDst selects the destination register (Rd for R-Type, Rt for I-Type, or R14 for JAL), and RegWrite enables writing while any attempt to write to R0 is ignored as it is hardwired to zero.

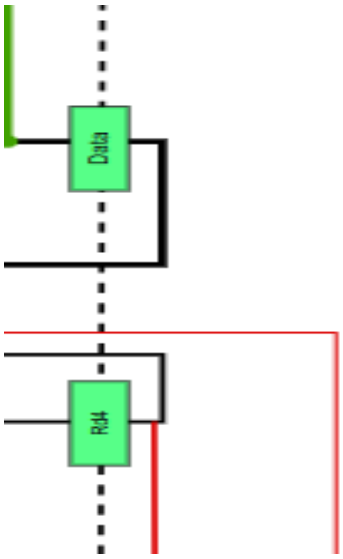


Figure 14 : WB Stage

2.6 Control Signals

Table 5: Control Signals

Signal	Description
RegWrite	Enable writing into register file
MemRead	Read data from memory
MemWrite	Write data into memory
ALUSrc	Select ALU second operand (Register/Immediate)
MemToReg	Select write-back source
Branch	Enable branch operation
Jump	Enable jump operation
ALUOp	Select ALU function
RegDst	Select destination register
Link	Used by JAL instruction

2.6.1 Control Truth Table

Table 6: Control Truth Table

Instruction	RegDst	ALUSrc	MemRead	MemWrite	MemToReg	RegWrite	Branch	Jump
ADD	1	0	0	0	0	1	0	0
SUB	1	0	0	0	0	1	0	0
AND	1	0	0	0	0	1	0	0
OR	1	0	0	0	0	1	0	0
XOR	1	0	0	0	0	1	0	0
NOR	1	0	0	0	0	1	0	0
SLL	1	0	0	0	0	1	0	0
SRL	1	0	0	0	0	1	0	0
CLR	1	0	0	0	0	1	0	0
ADDI	0	1	0	0	0	1	0	0
ANDI	0	1	0	0	0	1	0	0
ORI	0	1	0	0	0	1	0	0
LW	0	1	1	0	1	1	0	0
SW	X	1	0	1	X	0	0	0
SWAP	0	1	1	1	1	1	0	0
BEQ	X	0	0	0	X	0	1	0
BNE	X	0	0	0	X	0	1	0
J	X	X	0	0	X	0	0	1
JAL	X	X	0	0	X	1	0	1
JR	X	X	0	0	X	0	0	1

2.6.2 ALU Control

Table 7: ALU Control

Instruction	ALU Operation
ADD	ADD
SUB	SUB
AND	AND
OR	OR
XOR	XOR
NOR	NOR
SLL	Shift Left
SRL	Shift Right
CLR	Zero Output
ADDI	ADD
ANDI	AND
ORI	OR
LW	ADD
SW	ADD
SWAP	ADD
BEQ	SUB
BNE	SUB

2.7 Special Instructions

2.7.1 CLR Instructions

The CLR (Clear Register) instruction is a custom instruction used to reset the contents of a specified register to zero. During execution, the Control Unit enables register writing while the ALU generates a zero value that is written into the destination register. This instruction simplifies register initialization and is useful for clearing registers before performing calculations or storing new data. Since the operation does not require memory access, it is completed efficiently through the normal pipeline stages.

2.7.2 JR Instructions

The JR (Jump Register) instruction is a custom control-flow instruction that updates the Program Counter (PC) with the value stored in a source register. Unlike ordinary jump instructions that use an immediate target address, JR allows the jump destination to be determined dynamically at runtime. This instruction is particularly useful for implementing procedure returns and indirect jumps. When the instruction is decoded, the Control Unit activates the jump logic, and the PC is loaded with the contents of the specified register, causing execution to continue from the new address.

2.7.3 SWAP Instructions

The SWAP instruction is a custom memory operation that exchanges the contents of a register with a memory location in a single instruction. The effective memory address is calculated using the base register and immediate offset, similar to load and store instructions. During execution, the value stored in memory is read and written back to the register while the original register value is simultaneously written to the same memory location. This instruction combines load and store functionality into one operation, reducing the number of instructions required to exchange data and improving execution efficiency.

2.8 Boolean Equations

- **RegDst = ADD + SUB + AND + OR + XOR + NOR + SLL + SRL + CLR**
- **ALUSrc = ADDI + ANDI + ORI + LW + SW + SWAP**
- **MemToReg = LW + SWAP**
- **Link = JAL**

2.9 control-path diagrams

The Control Unit is responsible for generating all control signals required for processor operation. It is located in the Decode (ID) stage. In the Block Diagram in Figure 15, every red wire indicates a control path. Following is a description for each stage's control signals and what they are used for:

- **ID Stage:**
  - **RegDst :** Selects the destination register for write operations. For R-Type instructions, it selects the Rd field; for I-Type instructions, it selects the Rt field; for JAL, it selects R14.
  - **ALUSrc :** Selects the second operand for the ALU. When 0, the operand comes from the Rt register; when 1, the operand comes from the sign/zero-extended immediate value.
  - **Sign Or Zero :** Determines the extension type for the immediate value. Sign extension is used for arithmetic, memory, and branch instructions; zero extension is used for logical instructions (ANDI, ORI).

- **Branch :** Enables branch operations (BEQ and BNE). The ALU performs subtraction to evaluate the condition, and the PC is updated with the target address if the condition is true.

- **Jump :** Enables jump operations (J, JAL, and JR). For J and JAL, the PC is set to the target address; for JR, the PC is set to the value stored in the Rs register.

- **Link :** Used specifically by the JAL instruction. When asserted, the return address (PC + 1) is written into register R14.

- **EX Stage:**
  - **ALUOp :** Specifies the operation to be performed by the ALU. It selects between ADD, SUB, AND, OR, XOR, NOR, SLL, and SRL operations based on the instruction type.
- **MEM Stage:**
  - **MemRead :** Enables reading from the Data Memory. Asserted only for LW and SWAP instructions.
  - **MemWrite :** Enables writing to the Data Memory. Asserted only for SW and SWAP instructions.
- **WB Stage:**
  - **RegWrite :** Enables writing to the Register File. Asserted for instructions that produce a result (R-Type, ADDI, ANDI, ORI, LW, SWAP, JAL). Any attempt to write to R0 is ignored.
  - **MemtoReg :** Selects the source of data to be written back to the Register File. When 0, the ALU result is written; when 1, the memory read data is written (used by LW and SWAP).

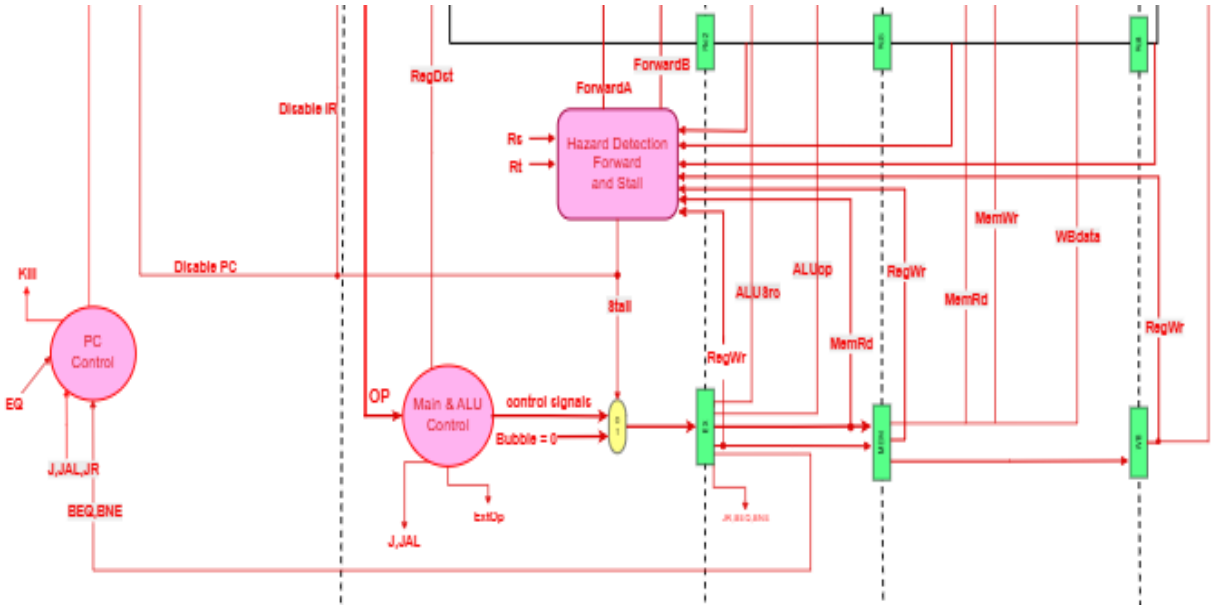


Figure 15 : control-path diagrams

2.10 Design Decisions & justification

- **Why Pipeline Was Selected**

A five-stage pipelined architecture was selected to improve processor performance and increase instruction throughput. Instead of waiting for one instruction to complete all stages before starting the next instruction, multiple instructions can be processed simultaneously in different pipeline stages. This approach allows the processor to execute instructions more efficiently and achieve higher performance compared to a non-pipelined implementation.

- **Why Separate Instruction and Data Memories Were Used**

Separate instruction memory and data memory were used to eliminate structural hazards between instruction fetch and data access operations. With separate memories, the processor can fetch an instruction and access data memory during the same clock cycle without conflicts. This design improves pipeline efficiency and simplifies hazard management.

- **How Hazards Were Handled**

The pipelined processor may encounter data hazards and control hazards during execution. Data hazards occur when an instruction depends on the result of a previous instruction. These hazards were handled using forwarding paths and pipeline stalling when necessary. Control hazards occur during branch and jump instructions because the next instruction address may not be known immediately. These hazards were handled by flushing incorrect instructions and updating the Program Counter once the branch decision was determined. Structural hazards were avoided through the use of separate instruction and data memories.

- **Advantages of the Design**

The proposed processor design provides several advantages:

- Increased instruction throughput through pipelining.
- Better utilization of processor hardware resources.
- Reduced execution time for instruction sequences.
- Support for arithmetic, logical, memory, branch, and jump instructions.
- Elimination of structural hazards through separate memories.
- Modular architecture that simplifies testing, debugging, and future extensions.

Overall, the pipelined architecture provides a balance between performance, simplicity, and scalability, making it suitable for implementing the required 32-bit RISC processor.

2.11 Simulation and Testing

2.11.1 Test Program 1: Arithmetic, Logical, Shift, Memory and SWAP Instructions

The first test program was designed to verify arithmetic operations, logical operations, shift instructions, memory access instructions, and the custom SWAP instruction. The program initializes registers using ADDI instructions and then performs several operations to verify the correct functionality of the ALU, Register File, Data Memory, and pipeline datapath.

Test Program

Figure 16 shows the assembly program used to test arithmetic, logical, shift, memory, and SWAP instructions.

```
// ============================================================
// Program 1: Arithmetic, logical, shift, memory, and SWAP
// ============================================================
$display("KERNEL: ===== Program 1: Required ISA Arithmetic/Logic/Memory/SWAP =====");
clear_imem();

write_imem(0,  I(6'd10, 4'd1, 4'd0, 18'd5));   // ADDI R1,R0,5
write_imem(1,  I(6'd10, 4'd2, 4'd0, 18'd10));  // ADDI R2,R0,10
write_imem(2,  R(6'd0,  4'd3, 4'd1, 4'd2));    // ADD R3,R1,R2 = 15
write_imem(3,  R(6'd1,  4'd4, 4'd2, 4'd1));    // SUB R4,R2,R1 = 5
write_imem(4,  R(6'd2,  4'd5, 4'd1, 4'd2));    // AND R5,R1,R2 = 0
write_imem(5,  R(6'd3,  4'd6, 4'd1, 4'd2));    // OR R6,R1,R2 = 15
write_imem(6,  R(6'd4,  4'd7, 4'd1, 4'd2));    // XOR R7,R1,R2 = 15
write_imem(7,  R(6'd6,  4'd6, 4'd1, 4'd1));    // SLL R6,R1,R1 = 160
write_imem(8,  R(6'd7,  4'd6, 4'd6, 4'd1));    // SRL R6,R6,R1 = 5
write_imem(9,  R(6'd8,  4'd5, 4'd0, 4'd0));    // CLR R5 = 0
write_imem(10, I(6'd14, 4'd1, 4'd0, 18'd0));   // SW R1,0(R0), MEM[0]=5
write_imem(11, I(6'd13, 4'd5, 4'd0, 18'd0));   // LW R5,0(R0), R5=5
write_imem(12, R(6'd0,  4'd6, 4'd5, 4'd1));    // ADD R6,R5,R1 = 10
write_imem(13, I(6'd15, 4'd2, 4'd0, 18'd0));   // SWAP R2,0(R0): R2=5, MEM[0]=10
write_imem(14, I(6'd13, 4'd5, 4'd0, 18'd0));   // LW R5,0(R0), R5=10
write_imem(15, HALT());

reset_processor();

repeat (40) begin
    #20 dump();
end

check("R0 hardwired zero", dbg_R0, 32'd0);
check("R1 ADDI", dbg_R1, 32'd5);
check("R2 after SWAP", dbg_R2, 32'd5);
check("R3 ADD", dbg_R3, 32'd15);
check("R4 SUB", dbg_R4, 32'd5);
check("R5 LW after SWAP", dbg_R5, 32'd10);
check("R6 load-use ADD", dbg_R6, 32'd10);
check("R7 XOR", dbg_R7, 32'd15);
check("MEM0 after SWAP", dbg_MEM0, 32'd10);
```

Figure 16 : Program 1 Source Code

Program Operations

The program performs the following operations:

- ADDI R1,R0,5
- ADDI R2,R0,10
- ADD R3,R1,R2
- SUB R4,R2,R1
- AND R5,R1,R2
- OR R6,R1,R2
- XOR R7,R1,R2
- SLL R6,R1,R1
- SRL R6,R6,R1
- CLR R5
- SW R1,0(R0)
- LW R5,0(R0)
- ADD R6,R5,R1
- SWAP R2,0(R0)
- LW R5,0(R0)

The following table summarizes the expected register and memory values after the successful execution of Program 1.

Expected Results

Register / Memory	Expected Value
R0	0
R1	5
R2	5
R3	15
R4	5
R5	10
R6	10
R7	15
MEM[0]	10

Simulation Waveform

Figure 17 shows the waveform generated during simulation after the execution of Program 1.

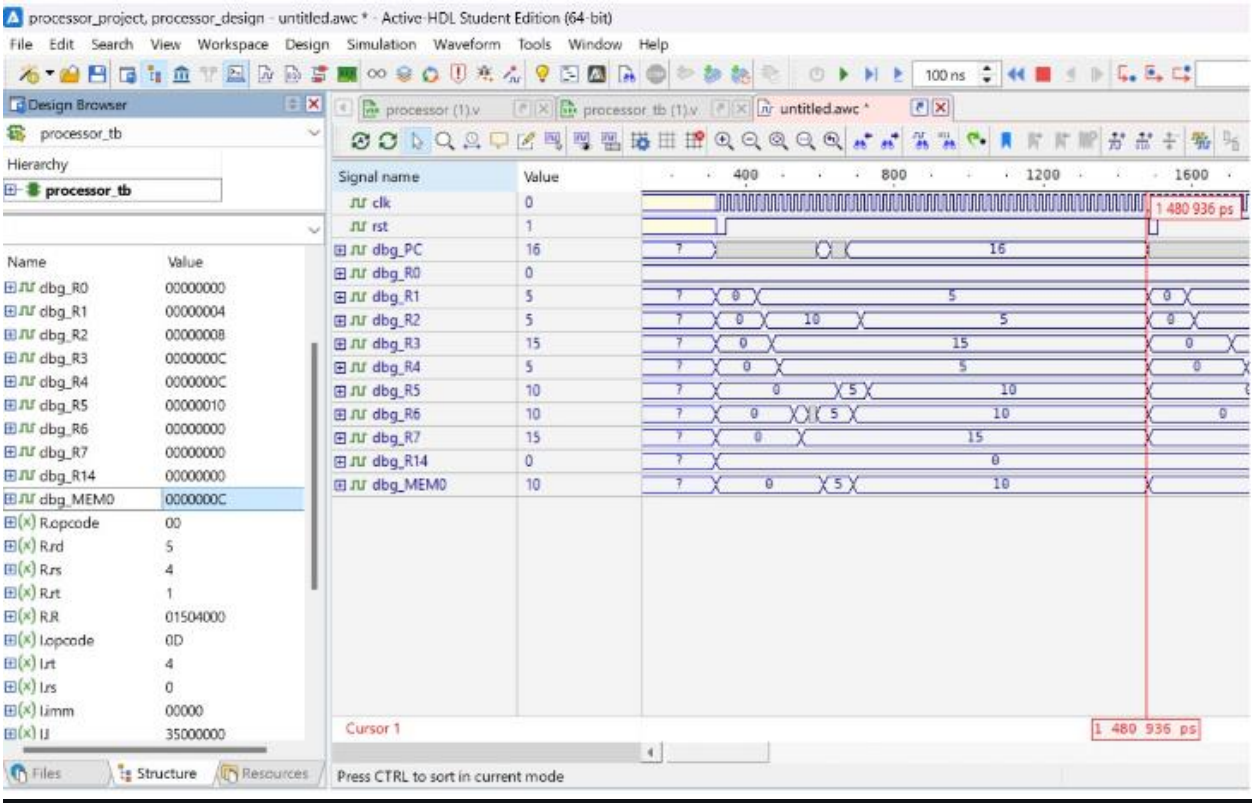


Figure 17 : Program 1 Waveform

The waveform confirms the correct execution of all arithmetic, logical, shift, memory, and SWAP instructions. Register values change exactly as expected throughout execution, and the final memory content matches the expected result.

Execution Trace

Figure 18 presents the simulation console output during execution.

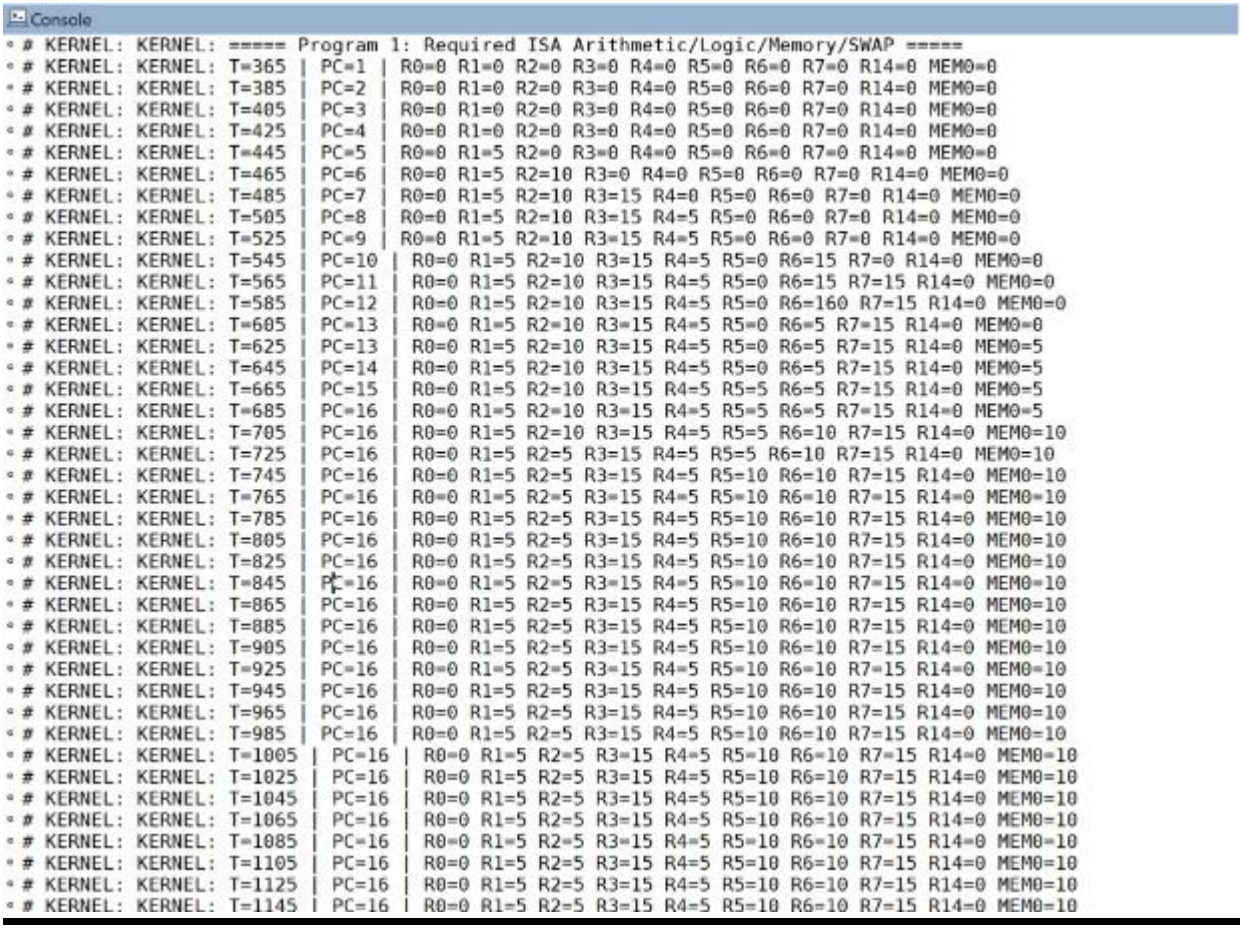


Figure 18 : Program 1 Execution Trace

The execution trace shows the processor state after each clock cycle, including the Program Counter, register contents, and memory values. The trace demonstrates the correct progression of instructions through the pipeline and con firms that all intermediate values are generated correctly.

Verification Results

Figure 19 presents the automatic verification results generated by the testbench.

```
# KERNEL: KERNEL: PASS: R0 hardwired zero = 0
# KERNEL: KERNEL: PASS: R1 ADDI = 5
# KERNEL: KERNEL: PASS: R2 after SWAP = 5
# KERNEL: KERNEL: PASS: R3 ADD = 15
# KERNEL: KERNEL: PASS: R4 SUB = 5
# KERNEL: KERNEL: PASS: R5 LW after SWAP = 10
# KERNEL: KERNEL: PASS: R6 load-use ADD = 10
# KERNEL: KERNEL: PASS: R7 XOR = 15
# KERNEL: KERNEL: PASS: MEM0 after SWAP = 10
```

Figure 19 : Program 1 PASS Results

The simulation results demonstrate that the processor correctly executes arithmetic, logical, shift, memory, and SWAP instructions. The observed register values and memory contents match the expected results, confirming the correctness of the ALU, Register File, Data Memory, and write-back operations. Furthermore, the successful PASS results verify that the datapath and control logic operate correctly throughout the pipeline.

The verification results confirm that all expected values were successfully obtained.

Checked Item	Result
R0 hardwired zero	PASS
R1 ADDI	PASS
R2 after SWAP	PASS
R3 ADD	PASS
R4 SUB	PASS
R5 LW after SWAP	PASS
R6 load-use ADD	PASS
R7 XOR	PASS
MEM[0] after SWAP	PASS

Discussion

The results demonstrate that the processor correctly executes arithmetic operations, logical operations, shift instructions, memory accesses, and the custom SWAP instruction. The waveform, execution trace, and automatic verification outputs all match the expected values, confirming the correctness of the datapath, control unit, ALU operations, memory subsystem, and write-back stage.

2.11.2 Test Program 2: Branch and Jump Instructions

The second test program was designed to verify all control-flow instructions implemented in the processor, including BEQ, BNE, J, JAL, and JR. These instructions modify the normal sequential execution of the program by changing the Program Counter (PC) based on branch conditions or jump targets.

Test Program

Figure 20 shows the assembly program used to verify branch and jump instructions.

```
168         // ============================================================
169         // Program 2: BEQ, BNE, J, JAL, JR
170         // ============================================================
171         $display("KERNEL: ===== Program 2: Branch + Jump + JAL + JR =====");
172         clear_imem();
173
174         write_imem(0,  I(6'd10, 4'd1, 4'd0, 18'd5));   // R1=5
175         write_imem(1,  I(6'd10, 4'd2, 4'd0, 18'd5));   // R2=5
176         write_imem(2,  I(6'd16, 4'd1, 4'd2, 18'd5));   // BEQ R1,R2,+5 -> PC=7
177         write_imem(3,  I(6'd10, 4'd3, 4'd0, 18'd99));  // skipped
178         write_imem(4,  I(6'd10, 4'd4, 4'd0, 18'd99));  // skipped
179         write_imem(5,  I(6'd10, 4'd5, 4'd0, 18'd99));  // skipped
180         write_imem(6,  I(6'd10, 4'd6, 4'd0, 18'd99));  // skipped
181         write_imem(7,  I(6'd10, 4'd3, 4'd0, 18'd7));   // R3=7
182         write_imem(8,  I(6'd17, 4'd1, 4'd2, 18'd3));   // BNE false, PC=9
183         write_imem(9,  I(6'd10, 4'd4, 4'd0, 18'd4));   // R4=4
184         write_imem(10, J(6'd18, 26'd3));                // J +3 -> PC=13
185         write_imem(11, I(6'd10, 4'd5, 4'd0, 18'd55));  // skipped
186         write_imem(12, I(6'd10, 4'd5, 4'd0, 18'd66));  // skipped
187         write_imem(13, J(6'd19, 26'd3));                // JAL +3 -> PC=16, R14=14
188         write_imem(14, J(6'd18, 26'd5));                // target after JR: J +5 -> PC=19
189         write_imem(15, I(6'd10, 4'd6, 4'd0, 18'd88));  // skipped
190         write_imem(16, I(6'd10, 4'd6, 4'd0, 18'd77));  // R6=77
191         write_imem(17, R(6'd9,  4'd0, 4'd14, 4'd0));   // JR R14 -> PC=14
192         write_imem(18, I(6'd10, 4'd6, 4'd0, 18'd99));  // skipped
193         write_imem(19, I(6'd10, 4'd5, 4'd0, 18'd20));  // R5=20
194         write_imem(20, HALT());
195
196         reset_processor();
197
198         repeat (55) begin
199             #20 dump();
200         end
201
202         check("R1", dbg_R1, 32'd5);
203         check("R2", dbg_R2, 32'd5);
204         check("R3 after BEQ target", dbg_R3, 32'd7);
205         check("R4 after BNE false", dbg_R4, 32'd4);
206         check("R5 after J/JR path", dbg_R5, 32'd20);
207         check("R6 after JAL target", dbg_R6, 32'd77);
208         check("R14 link register", dbg_R14, 32'd14);
209
```

Figure 20: Program 2 Source Code

Program Operations

The program verifies the following operations:

- BEQ branch taken.
- BNE branch not taken.
- J unconditional jump.
- JAL jump and link.
- JR jump register.
- Link register update (R14).

The following table presents the expected register values after executing all branch and jump instructions.

Expected Results

Register	Expected Value
R1	5
R2	5
R3	7
R4	4
R5	20
R6	77
R14	14

Simulation Waveform

Figure 21 shows the waveform generated during simulation after the execution of Program 2.

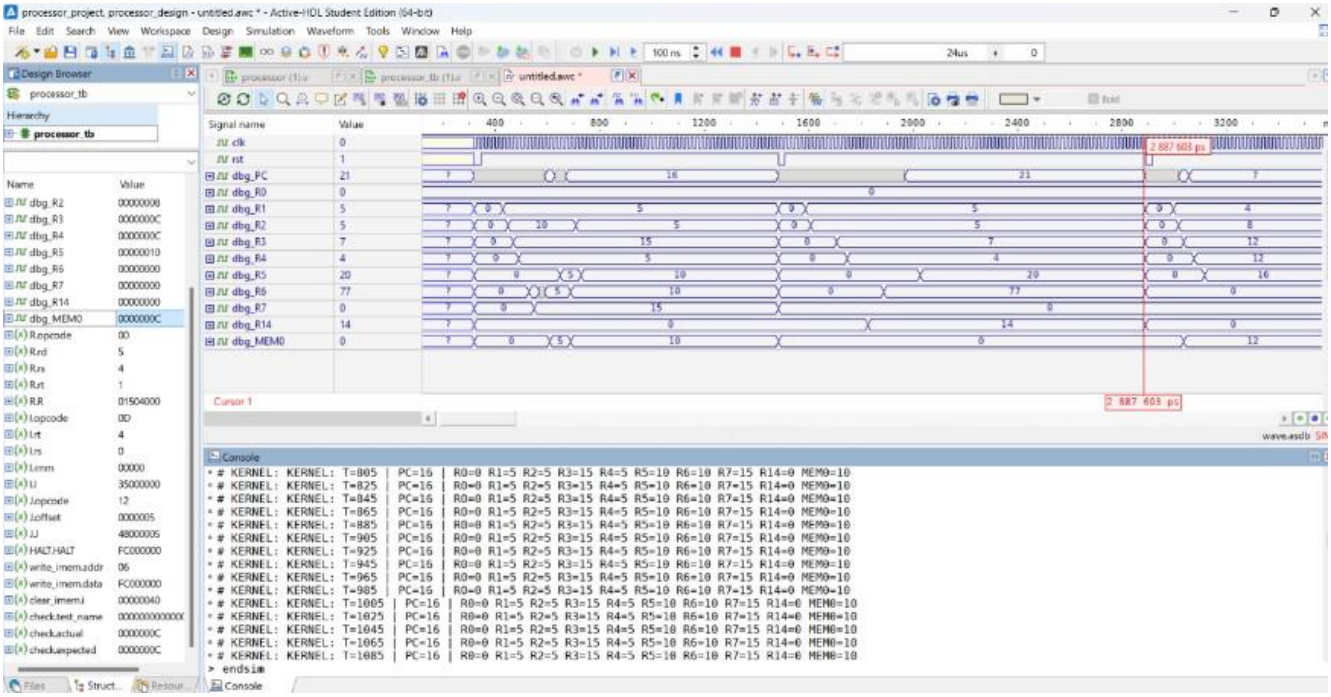


Figure 21: Program 2 Waveform

The waveform confirms the correct execution of all control-flow instructions implemented in the processor. The Program Counter (PC) changes according to the behavior of each instruction, where **BEQ** successfully branches when the condition is true, **BNE** continues sequential execution when the condition is false, **J** performs an unconditional jump, **JAL** stores the return address in register **R14**, and **JR** correctly jumps to the address stored in the register. The waveform also verifies that the pipeline handles branch and jump operations correctly while maintaining proper instruction flow and register updates.

Execution Trace

Figure 21 shows the execution trace generated during simulation.

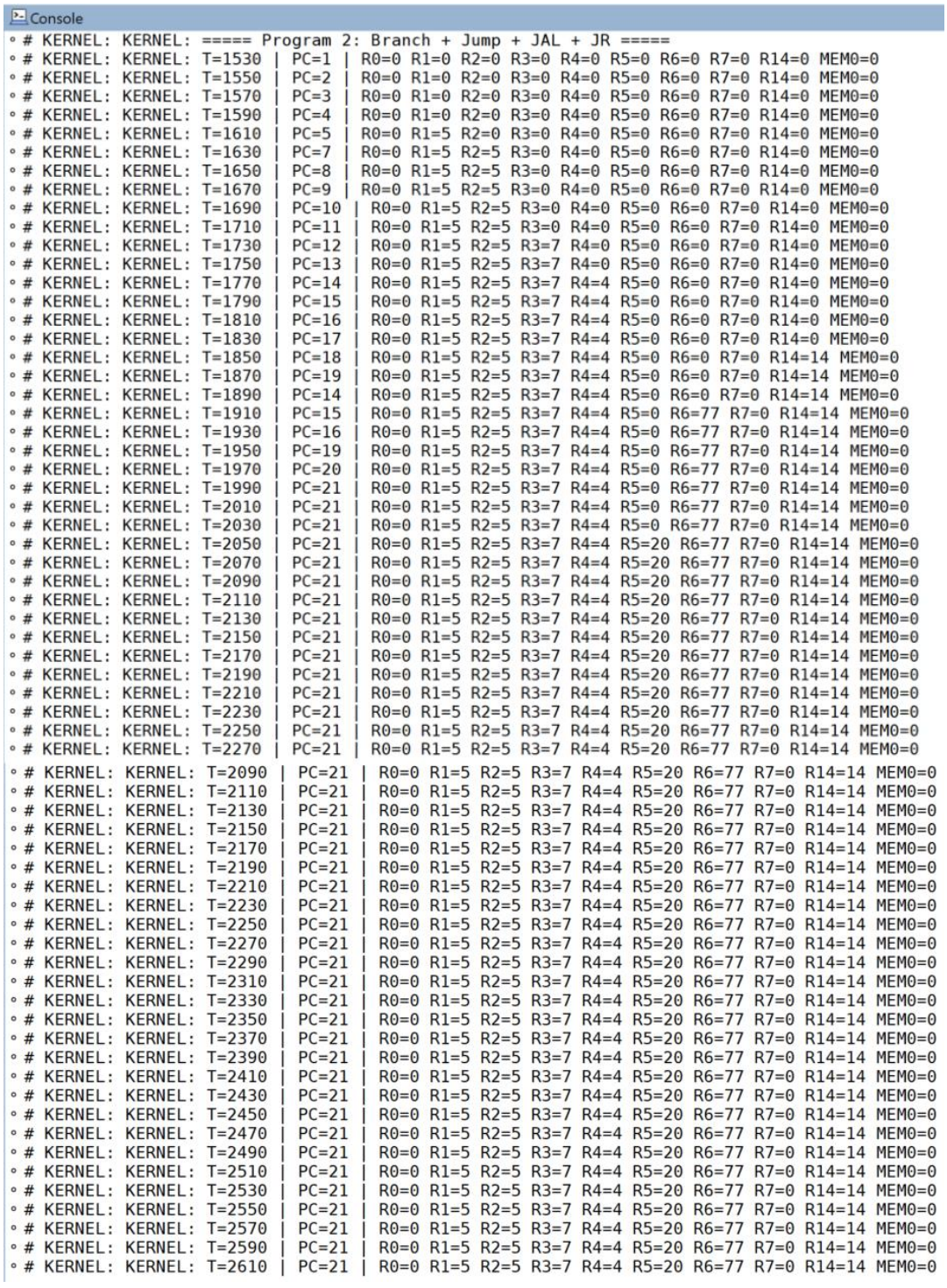


Figure 22: Program 2 Execution Trace

The execution trace demonstrates correct control-flow behavior. The BEQ instruction successfully branches when the comparison condition is satisfied, while the BNE instruction continues normal execution because the condition is false. The J instruction performs an unconditional jump, JAL correctly stores the return address in register R14, and JR successfully returns execution to the address stored in R14.

Verification Results

Figure 22 presents the automatic verification results generated by the testbench.

```
# KERNEL: KERNEL: PASS: R1 = 5
# KERNEL: KERNEL: PASS: R2 = 5
# KERNEL: KERNEL: PASS: R3 after BEQ target = 7
# KERNEL: KERNEL: PASS: R4 after BNE false = 4
# KERNEL: KERNEL: PASS: R5 after J/JR path = 20
# KERNEL: KERNEL: PASS: R6 after JAL target = 77
# KERNEL: KERNEL: PASS: R14 link register = 14
```

Figure 23: Program 2 PASS Results

Checked Item	Result
R1 Initialization	PASS
R2 Initialization	PASS
R3 after BEQ target	PASS
R4 after BNE false path	PASS
R5 after J/JR path	PASS
R6 after JAL target	PASS
R14 link register	PASS

Discussion

The simulation results verify the correct behavior of all implemented control-flow instructions. The processor correctly updates the Program Counter during BEQ, BNE, J, JAL, and JR instructions. In addition, the link register (R14) is updated correctly by the JAL instruction, confirming the proper operation of the control unit, jump logic, branch logic, and pipeline control.

2.11.3 Test Program 3: Forwarding and Load-Use Hazard

The third test program was designed to verify the forwarding unit and load-use hazard handling in the pipelined processor. This test is important because it checks whether the processor can correctly execute dependent instructions without producing incorrect results.

Test Program

Figure 23 shows the assembly program used to test forwarding and load-use hazard handling.

```
// ============================================================
// Program 3: Forwarding and load-use hazard
// ============================================================
$display("KERNEL: ===== Program 3: Forwarding + Load-Use Hazard =====");
clear_imem();

write_imem(0, I(6'd10, 4'd1, 4'd0, 18'd4));    // R1=4
write_imem(1, R(6'd0,  4'd2, 4'd1, 4'd1));     // R2=R1+R1=8
write_imem(2, R(6'd0,  4'd3, 4'd2, 4'd1));     // R3=R2+R1=12
write_imem(3, I(6'd14, 4'd3, 4'd0, 18'd0));    // MEM[0]=12
write_imem(4, I(6'd13, 4'd4, 4'd0, 18'd0));    // R4=MEM[0]=12
write_imem(5, R(6'd0,  4'd5, 4'd4, 4'd1));     // R5=R4+R1=16
write_imem(6, HALT());

reset_processor();

repeat (35) begin
    #20 dump();
end

check("R1", dbg_R1, 32'd4);
check("R2 forwarding", dbg_R2, 32'd8);
check("R3 forwarding", dbg_R3, 32'd12);
check("R4 load", dbg_R4, 32'd12);
check("R5 load-use hazard result", dbg_R5, 32'd16);
check("MEM0 store", dbg_MEM0, 32'd12);

$display("KERNEL: ===== Simulation Complete =====");

// The test sequence finishes around 3619 ns.
// This delay extends the waveform to about 24 us total.
// 24000 ns - 3619 ns = 20381 ns.
#20381;
```

Figure 24 : Program 3 Source Code

Program Operations

The program performs the following operations:

Instruction	Purpose
ADDI R1,R0,4	Initialize R1 with 4
ADD R2,R1,R1	Tests forwarding, R2 = 8
ADD R3,R2,R1	Tests forwarding again, R3 = 12
SW R3,0(R0)	Store R3 into memory
LW R4,0(R0)	Load memory value into R4
ADD R5,R4,R1	Tests load-use hazard, R5 = 16

The following table summarizes the expected register and memory values used to verify forwarding and load-use hazard handling.

Expected Results

Register / Memory	Expected Value
R1	4
R2	8
R3	12
R4	12
R5	16
MEM[0]	12

Simulation Waveform

Figure 24 shows the waveform generated during the execution of Program 3.

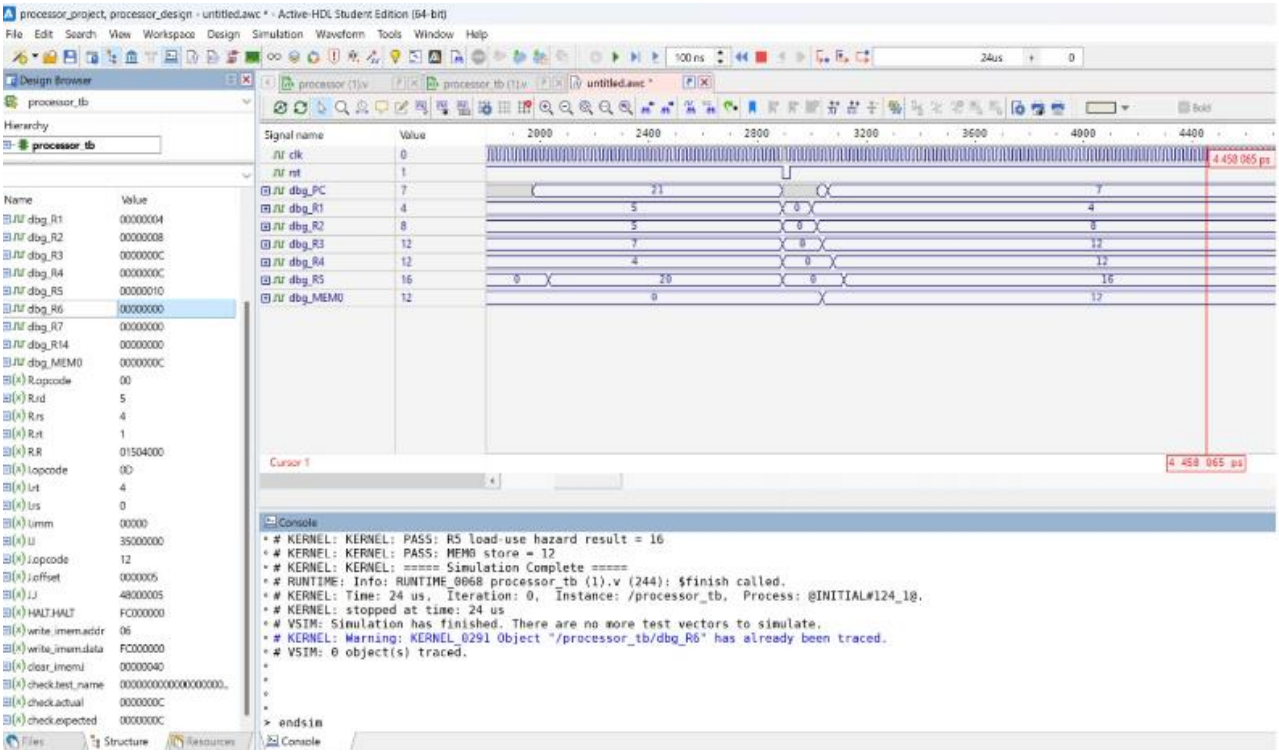


Figure 25 : Program 3 Waveform

The waveform confirms that the dependent instructions were executed correctly. The values of R2 and R3 prove that forwarding worked successfully, while the correct value of R5 confirms that the load-use hazard was handled properly.

Execution Trace

Figure 25 shows the simulation console trace for Program 3.

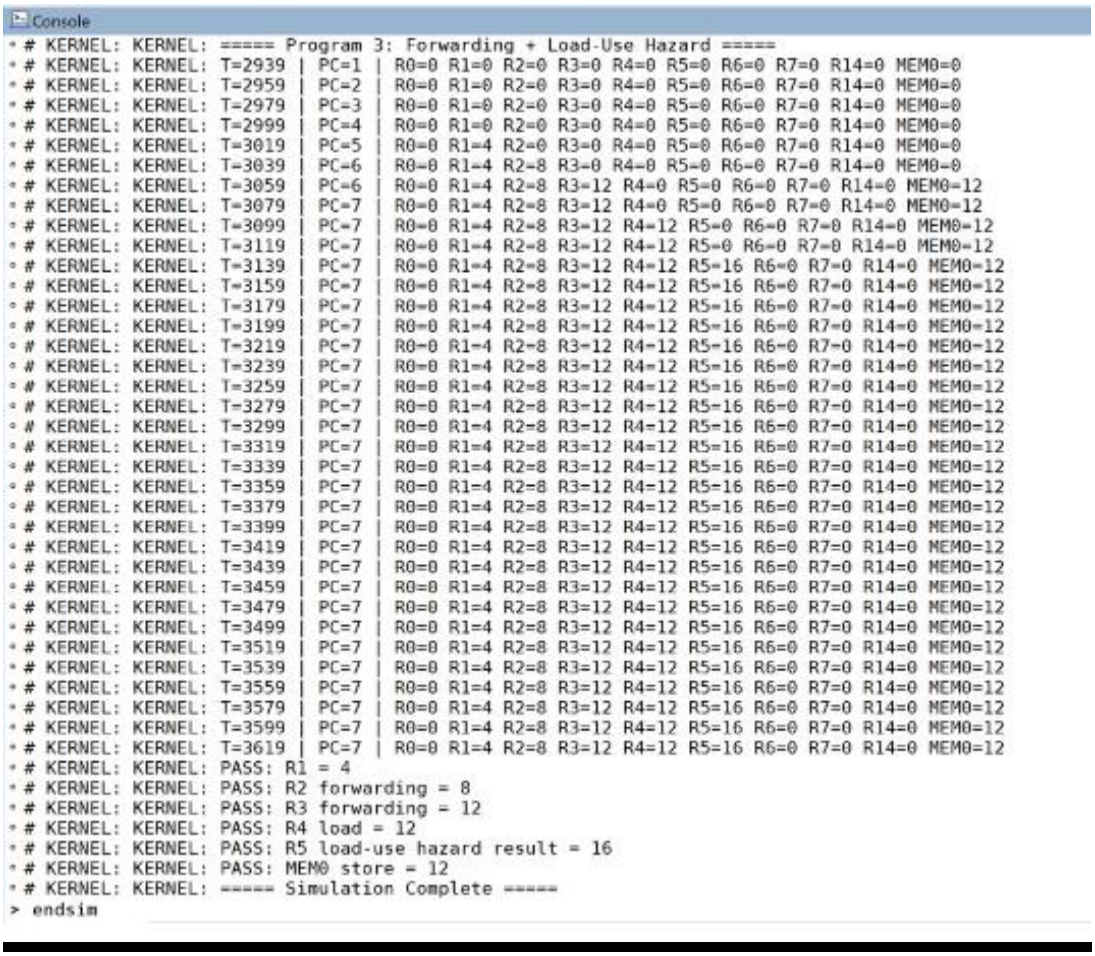


Figure 26 : Program 3 Execution Trace

The execution trace shows the cycle-by-cycle behavior of the processor. The final values confirm that the processor correctly forwarded values between dependent instructions and correctly handled the load-use dependency after the LW instruction.

Verification Results

Figure 26 presents the automatic PASS results generated by the testbench.

```
# KERNEL: KERNEL: PASS: R1 = 4
# KERNEL: KERNEL: PASS: R2 forwarding = 8
# KERNEL: KERNEL: PASS: R3 forwarding = 12
# KERNEL: KERNEL: PASS: R4 load = 12
# KERNEL: KERNEL: PASS: R5 load-use hazard result = 16
# KERNEL: KERNEL: PASS: MEM0 store = 12
# KERNEL: KERNEL: ===== Simulation Complete =====
```

Figure 27 : Program 3 PASS Results

Checked Item	Result
R1	PASS
R2 forwarding	PASS
R3 forwarding	PASS
R4 load	PASS
R5 load-use hazard result	PASS
MEM[0] store	PASS

Discussion

The obtained results confirm the correct operation of both the forwarding unit and the hazard detection unit. Data dependencies between consecutive instructions were resolved through forwarding whenever possible, while the load-use hazard was handled correctly by inserting the required stall cycle. Consequently, all final register and memory values matched the expected results, verifying the correctness of the implemented pipeline hazard handling.

2.11.4 Overall Testing Summary

The three test programs collectively verified all required processor instructions, including arithmetic, logical, shift, memory, branch, jump, and custom instructions. All automatic testbench checks passed successfully, confirming that the implemented processor executes the required ISA correctly.

Instruction Category	Instructions Verified
Arithmetic	ADD, SUB, ADDI
Logical	AND, OR, XOR, NOR, ANDI, ORI
Shift	SLL, SRL
Memory	LW, SW
Custom	CLR, SWAP, JR
Branch	BEQ, BNE
Jump	J, JAL

2.12 Conclusion

This project successfully designed and implemented a complete 32-bit pipelined RISC processor using Verilog HDL based on a five-stage pipeline architecture consisting of Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB). The processor supports a comprehensive instruction set including arithmetic, logical, memory access, branch, jump, and custom instructions such as CLR, JR, and SWAP. The design incorporates all major processor components, including the Program Counter, Instruction Memory, Register File, ALU, Data Memory, Control Unit, Sign/Zero Extension Unit, Hazard Detection Unit, and Forwarding Unit, enabling efficient and correct instruction execution. To improve performance, separate instruction and data memories were utilized to eliminate structural hazards, while forwarding and hazard detection mechanisms were implemented to minimize pipeline stalls and resolve data dependencies. The processor was thoroughly verified through simulation and assembly test programs covering all supported instructions, and the obtained results confirmed the correct operation of Datapath components, control signals, memory operations, branching, jumping, hazard handling, and write-back functionality. Overall, the project met all specified requirements and provided valuable experience in processor architecture design, pipelined microarchitecture implementation, hardware verification, and digital system development using Verilog HDL.

2.13 References

[1] Arithmetic Logic Definition in Neuroscience. Available at: https://ineurociencias.org/arithmetic-logic-definition-in-neuroscience/. Accessed on: June 25, 2026 at 12:20 PM.

[2] All Wiring Diagram, “Circuit Diagram For Program Counter,” Apr. 2020. [Online]. Available: https://wiringdiagramall.blogspot.com/2020/04/circuit-diagram-for-program-counter.html. Accessed: June 25, 2026, 12:35 PM.

[3] ENCS4370 Computer Architecture Course Slides, Department of Electrical and Computer Engineering, Birzeit University, Spring 2025/2026.

2.14 Teamwork

• Design Process and Processor Architecture:

- The processor architecture and datapath were designed collaboratively by all team members. The instruction set, datapath organization, pipeline stages, and control-path were discussed together before implementation. Each member contributed ideas to improve the overall processor design and verify its correctness.

• Implementation in Verilog:

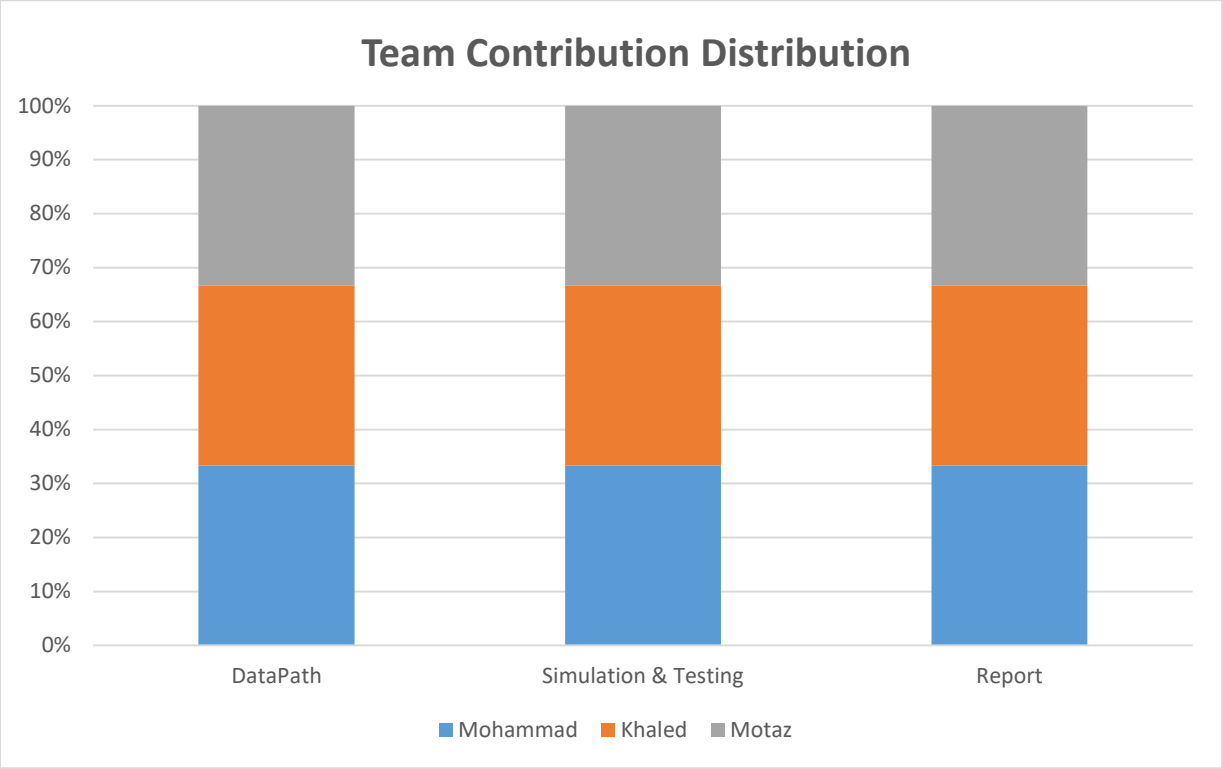
- **Mohammad:** Implemented the Program Counter (PC), Instruction Memory, and pipeline registers. He also participated in integrating the processor modules.
- **Khaled:** Implemented the ALU, Register File, Data Memory, Control Unit, Hazard Detection Unit, and Forwarding Unit. He also assisted in debugging and module integration.
- **Motaz:** Implemented the remaining datapath components, including the Sign/Zero Extension Unit and branch/jump logic. He also participated in integrating and verifying the complete processor.

• Simulation and Testing:

- **Mohammad:** Prepared and verified Test Program 1 (Arithmetic, Logical, Memory, and SWAP instructions).
- **Khaled:** Prepared and verified Test Program 2 (Branch, Jump, JAL, and JR instructions).
- **Motaz:** Prepared and verified Test Program 3 (Forwarding and Load-Use Hazard) and assisted in validating the final simulation results.

• Report Writing:

- **Mohammad:** Wrote the Introduction, Processor Specifications, Processor Architecture, and Conclusion sections.
- **Khaled:** Wrote the Pipeline Architecture, Control Signals, Special Instructions, Boolean Equations, and Design Decisions sections.
- **Motaz:** Wrote the Simulation and Testing sections, organized the figures and tables, formatted the report, prepared the references, and reviewed the final document before submission.


Team Contribution Distribution
100%
90%
80%
70%
60%
50%
40%
30%
20%
10%
0%
DataPath
Simulation & Testing
Report
Mohammad
Khaled
Motaz